

Projet Algorithmie 3

La distance de Jaccard

De-backer Cuvelier Sébastien Llobregat Nicolas

Version du 24 Avril 2025

Table des matières

1 Organisation du travail	2
1.1 Outils utilisés	2
1.2 Répartition du travail	2
2 Structure du projet	2
2.1 Les modules	2
2.2 Gestion des options	3
2.3 Gestions des fichiers	4
2.4 Gestion de l'entrée standard	4
3 Choix d'implémentation	5
3.1 Stockage des mots avec holdall	5
3.1.1 Implémentation par LDSC	5
3.1.2 Implémentation par tableaux dynamiques	5
4 Calcul de la distance de Jaccard :	5
5 Les difficultés rencontrées	7
5.1 Tentative d'utilisation de getopt	7
5.2 La fonction fscanf	7
6 Performances	7
7 Améliorations possibles	9

1 Organisation du travail

1.1 Outils utilisés

Git & Github : Pour la gestion du projet et des différentes versions de celui-ci nous avons fait le choix d'utiliser un logiciel de versionnage : **GIT**. Ainsi nous avons pu travailler en simultané sur les différentes composantes du projet. La méthodologie était la suivante : Créations des tâches via les "issues" de **Github**. Puis création d'une branche spécifique pour chaque tâche à réaliser. Une fois le travail réaliser nous ouvrons une **pull request** pour que notre binôme puisse analyser le travail de l'autre avant de mutualiser sur la branche principale.

Tests unitaires : Nous avons pour ambition d'utiliser les tests unitaires pour s'assurer que chaque fonction réalisait bien ce qu'elle devait, pour se faire nous avons commencer à utiliser une bibliothèque qui permettait assez simplement de les réalilser: **greatest**. Cependant, il s'est avéré très chronophage et nous avons pris du retard sur ces tests ce qui leur a fait perdre toute utilité. Nous avons donc choisi de ne pas les inclure dans notre rendu final du projet.

1.2 Répartition du travail

Mesure de la charge de travail : Au début du projet, nous n'avions pas réellement conscience de la charge de travail à effectuer. Il aurait peut-être été intéressant de lors de la première séance de réaliser un planning avec un tableau de répartition des tâches pour définir une charge de travail équitable. Cependant, le projet n'étant pas de grande envergure et étant en effectif réduis à deux personnes nous avons pu communiquer efficacement pour communiquer sur les tâches que nous étions réalisions.

2 Structure du projet

Origine : Lors de la conception de notre binôme nous avons décidé de faire une lecture commune du sujet afin de savoir si nous avions la même vision. Ainsi nous avons organisé la structure du projet à l'origine puis nous nous sommes mis d'accord sur les modules à utiliser. Entre les deux implémentations suggérées soit l'utilisation de **tables de hachages** ou bien **d'arbres binaires de recherches équilibrés**, c'est la deuxième qui nous a convaincu. En effet, il est plus facile de représenter les arbres et nous avons beaucoup plus travaillé sur cette notion en cours.

2.1 Les modules

Les modules utilisés sont: **avl**, **jdis**.

Le module jdis : Nous avons décidé de créer un module spécifique pour gérer la liaison entre le module **avl** et notre utilisation, il consiste principalement à convertir un fichier ou l'entrée standard (voir **2.4**) en un **avl**. En le calcul du cardinal de l'intersection des ensembles de mots stockés dans les arbres. Et bien évidemment au calcul de la distance de Jaccard entre chaque couple deux à deux distincts des ensembles représentés par les arbres sur laquelle nous reviendrons dans la sous section **4**.

2.2 Gestion des options

Généralités : Pour ce qui est de la gestion des options, nous avons décidé d’implanter à la fois les options longues et les options courtes. Les paragraphes qui suivent préciseront ce que ces options font et comment elles le font.

L’option -? / -help : Cette option est très simple, elle permet d’afficher le menu d’aide pour l’utilisation de notre exécutable et affiche par conséquent des précisions sur les options supportées.

L’option -g / -graph : Cette option permet d’afficher un graphe d’appartenance des mots dans les différents fichiers passés sur la ligne de commande. Cette fonction est divisée en deux parties.

```
1 static int scptr_display(context *ctx, const char *ref);
```

Cette fonction permet d’afficher une ligne du graph, pour se faire le processus est assez simple, nous allons itérer sur les arbres présents dans le tableau contenu lui-même dans le contexte qui est pris en paramètre. Ainsi nous allons rechercher la présence de chaque mot présent dans l’arbre de l’union des fichiers dans les arbres, le cas échéant nous affichons sur la colonne ‘x’ indiquant que le fichier contient la référence sinon ‘-’. Cette fonction est alors utilisée dans la fonction principale et non statique :

```
1 int graph_belonging(bst **t, bst *uni, int nb_file);
```

Celle-ci va utiliser la fonction **bst_dft_infix_apply_context** du module **avl** pour appliquer le traitement à chaque mot de l’arbre des unions.

L’option -iVALUE / -initial=VALUE : Cette option permet de fixer la taille des mots à **VALUE**. Celle-ci est traitée au moment de la lecture des mots et permet de signaler qu’il faut ignorer tous les caractères dépassant cette limite jusqu’au prochain caractère de séparation (voir 5.2). À savoir que par défaut il n’y a pas de limite en tant que telle, la taille maximale d’un mot sera alors **ULONG_MAX** ce qui est plus que raisonnable.

L’option -p / -punctuation-like-space : Cette option permet d’indiquer à la fonction de lecture que les caractères de ponctuation de la classe `isspace` et `ispunct` doivent être considérés comme des séparateurs pour segmentation des mots.

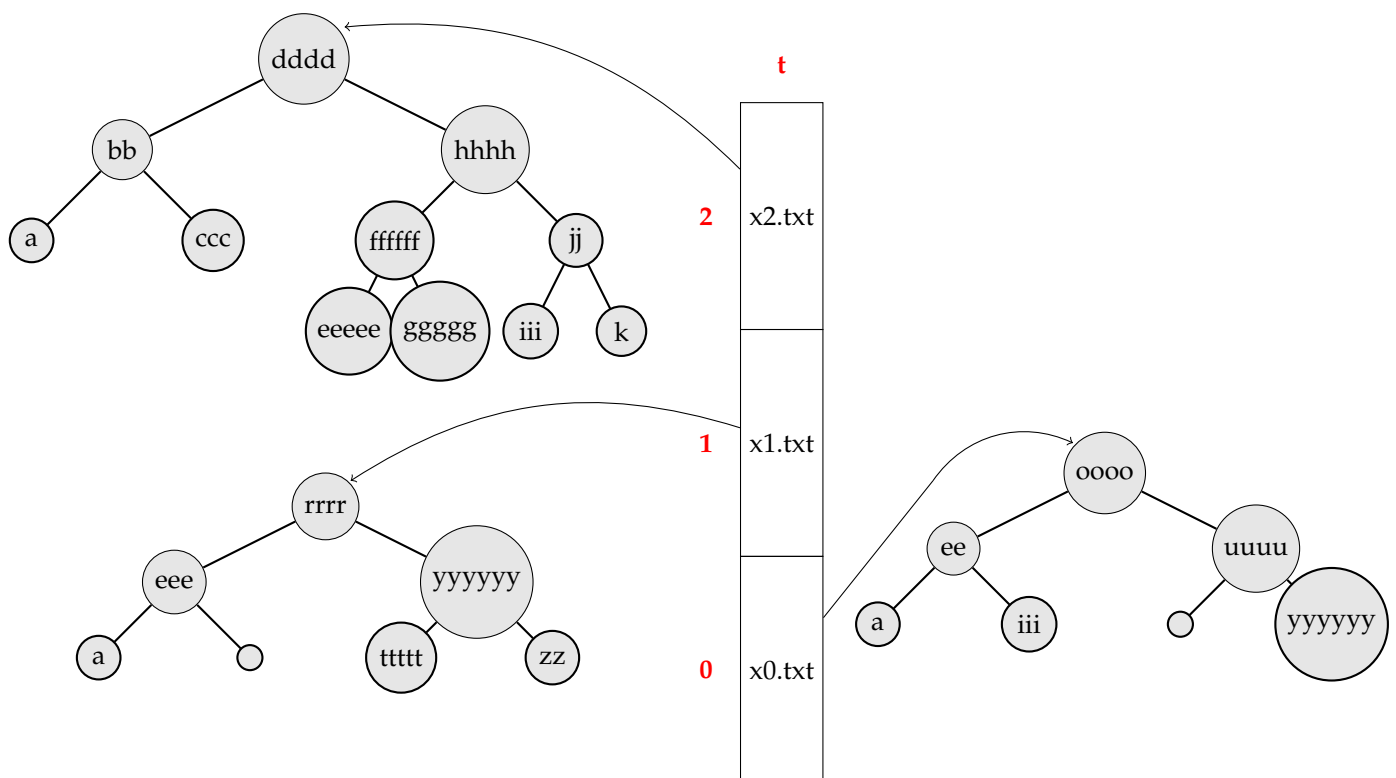
L’option -- : Cette option permet d’échapper l’argument qui suit sur la ligne de commande. Si nous avons un fichier appelé `--help`, par exemple, cette option nous permet de traiter ce fichier comme un fichier et non comme l’option `-help` ci-dessus. Cela se fait tout simplement par le fait d’effectuer le test de comparaison de la présence de l’option en premier afin de stocker le nom du fichier.

L’option - : Cette dernière option permet de rentrer des mots sur l’entrée standard pour qu’ils soient traités comme un fichier (voir 2.4).

2.3 Gestions des fichiers

Pour gérer les fichiers pris en arguments par le programme nous ouvrons simplement le fichier associé en mode lecture seule puis nous lisons (précisions 5.2) chaque mot et nous tentons de l'ajouter à l'arbre binaire associé. À la fin de la lecture, nous avons un arbre contenant des pointeurs vers les mots. Il est à noter que pour un même fichier, on stocke en un et un seul exemplaire les mots apparaissant plusieurs fois dans le même fichier. Nous avons un tableau avec l'ensemble des arbres créés.

Exemple : Ainsi si nous exécutons la commande suivante : `./jdis x0.txt x1.txt x2.txt` où les fichiers x0.txt et x1.txt sont ceux donnés dans le sujet du projet, nous aurons la représentation suivante :



Exemple 1: Conversion d'un fichier en arbre

2.4 Gestion de l'entrée standard

Questionnement : Nous avons réfléchi comment traiter l'argument particulier '-', deux Choix s'offraient alors à nous, créer l'arbre depuis l'entrée standard directement ou bien de stocker l'entrée standard dans un fichier puis effectuer le même traitement que sur les autres arguments sans distinction aucune. Par soucis d'efficacité, c'est donc la première solution qui a été choisie. Dans le cas où l'on détecte '-' à la place d'un nom de fichier alors nous avons plus qu'à lire depuis l'entrée standard.

3 Choix d'implémentation

3.1 Stockage des mots avec holdall

Généralités : Dans un premier temps nous avons eu recours au module **holdall** pour le stockage des mots, afin de séparer le traitement des mots de la gestion en mémoire. Nous aurions pu choisir d'utiliser la fonction **bst.dft_infix_apply_cont** du module **avl** pour libérer les ressources. Nous avons également décidé de tester cette solution en parallèle et nous en faisons référence dans la section 6 liée aux performances de l'implémentation proposée. Deux versions de ce module ont été réfléchies pendant la création de ce projet. La première en utilisant le principe des listes dynamiques simplement chaînées et la seconde en utilisant les tableaux dynamiques.

3.1.1 Implémentation par LDSC

Pendant les cours d'algorithmique, nous avons étudié les listes dynamiques simplement chaînées (LDSC). Ce type de structure permet d'ajouter rapidement des éléments en tête de liste, en temps constant, ce qui est très avantageux en termes de performance. En effet, chaque ajout nécessite juste de créer un nouveau maillon et de modifier un pointeur, sans avoir à déplacer d'autres éléments. Cependant, cette rapidité a un coût : chaque maillon prend plus de mémoire, car il contient un pointeur en plus de la donnée. Cela devient encombrant quand les fichiers sont volumineux. De plus, l'accès à un élément précis est plus lent, car il faut parcourir la liste depuis le début.

3.1.2 Implémentation par tableaux dynamiques

Pour résoudre le problème d'espace, nous avons pensé aux tableaux dynamiques. En effet l'avantage, ici, est que les ajouts se font également en temps constant grâce à un accès direct aux cellules du tableau et nous n'avons pas besoin d'allouer toute une cellule de liste pour un seul mot ; nous avons juste à allouer un tableau assez grand pour stocker les mots. Pour les petits fichiers cette implémentation s'est montrée très efficace, mais pour les gros fichiers le programme passait énormément de temps à recopier les données déjà présentes dans le tableau.

Conclusion : Nous avons finalement décidé de ne pas utiliser ce module pour avoir une performance optimale sur notre exécutable. Néanmoins, nous avons réalisé une petite étude sur un petit jeu de données afin de se rendre compte des différences.

4 Calcul de la distance de Jaccard :

Les paires distinctes : Nous avons maintenant notre tableau avec des pointeurs vers nos arbres, on peut rappeler que la taille d'un arbre s'effectue en temps constant dans le contrôleur. Nous allons avoir à calculer la distance de Jaccard entre chaque couple deux à deux distincts d'arbres. Pour cela il suffit d'avoir deux boucles imbriquées :

```

1  for (int i = 0; i < k - 1; ++i) {
2      for (int j = i + 1; j < k; ++j) {
3          . . .
4      }
5  }

```

La première boucle : nous commençons à $i = 0$ et allons jusqu'à $i = k - 2$ (car $i < k - 1$)

La deuxième boucle : pour chaque valeur de i nous commençons à $j = i + 1$ et va jusqu'à $j = k - 1$ (car $j < k$). L'idée ici est de parcourir chaque combinaison unique de i et j où i est strictement inférieur à j . Cela permet de créer une paire unique sans répéter les mêmes éléments dans les deux positions.

Rappel sur la distance de Jaccard : Nous devons maintenant effectuer le calcul de la distance de jaccard, petit rappel sur la formule : La **distance de Jaccard** entre deux ensembles A et B est définie par la formule suivante :

$$J_{\delta}(A, B) = \begin{cases} 0 & \text{si } A = \emptyset \wedge B = \emptyset, \\ 1 - \frac{|A \cap B|}{|A \cup B|} & \text{sinon.} \end{cases}$$

Calcul du cardinal de l'union : Nous avons donc besoin du cardinal de l'intersection ainsi que le cardinal de l'union de chaque couple, pour cela nous allons en réalité ne calculer que le cardinal de l'intersection. En effet, nous allons utiliser une petite astuce de la théorie des ensembles qui nous dit que : Le cardinal de l'union de deux ensemble ce n'est rien d'autre que la somme de leurs cardinaux respectifs moins le cardinal de l'intersection. On peut facilement s'en rendre compte ici:

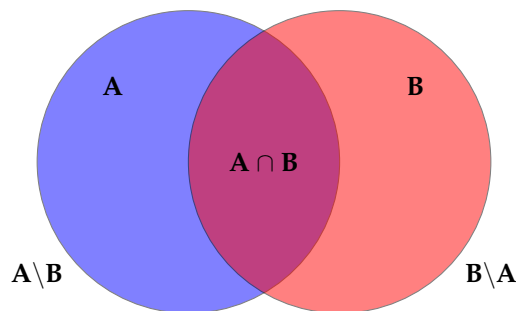


Schéma 1: Généralités sur les ensembles

Calcul du cardinal de l'intersection Nous devons alors calculer l'intersection des deux ensembles, pour cela nous allons de nouveau utiliser la fonction `bst.dft.infix.apply.context` du module `avl`. Cette fois-ci nous allons un contexte qui sera une structure contenant un compteur ainsi que le plus petit des deux ensembles. Ainsi, nous allons parcourir le plus grand des deux et rechercher dans le plus petit la référence grâce à la fonction `bst.search` du module `avl`. Si on la trouve alors on ajoute un au compteur cela veut dire qu'elle se trouve dans les deux sinon non. À la fin dans le contexte, nous aurons le cardinal de l'ensemble.

Calcul final : Il nous reste plus qu'à rassembler les éléments, ainsi nous avons : Un moins le cardinal de l'intersection divisé par la taille des deux arbres moins le cardinal de l'intersection ce qui nous donne la distance de Jaccard entre les deux ensembles de mots de la paire de fichiers. On peut alors réitérer sur toutes les paires.

5 Les difficultés rencontrées

5.1 Tentative d'utilisation de `getopt`

La bibliothèque `getopt` : En premier lieu nous nous sommes intéressés à la bibliothèque `getopt` qui nous permettait de parser la ligne de commande pour trouver les options notamment grâce à **`optind`** et **`optarg`**. Nous pouvions également préciser quelles options nécessitaient des paramètres comme pour l'option qui permet de fixer la taille maximale des mots **`-iVALUE`** ou **`-initial=VALUE`**. Grâce à **`getopt`** nous avons pu implanter les options très rapidement, mais nous avons rencontré un problème lorsque nous voulions implanter l'option **`'-'`** permettant d'échapper le nom d'un fichier. En effet, lorsque nous utilisions cette option, nous avons remarqué qu'elle échappait tous les paramètres qui suivaient. Nous avons d'abord cherché une solution en remplaçant **`getopt`** par **`getopt.long`**. Ce qui nous compliquait la tâche. Finalement il a été choisi de parser nous même la ligne de commande pour avoir un contrôle total sur son traitement.

5.2 La fonction `fscanf`

utilisation de `fscanf` : Afin de créer les arbres comme nous l'avons décrit dans la section 2.3 nous avons besoin de lire dans les fichiers, pour se faire il convient d'usage d'utiliser la fonction **`fscanf`**, et c'est le choix que nous avons fait lors de la première partie du projet cependant, après avoir essayé d'implémenter l'option **`-p`** (resp. **`-punctuation-like-space`**) nous avons un problème de compréhension de la fonction. Il fallait de plus analyser une deuxième fois le buffer que nous prenions afin de l'analyser par la suite. En outre cela revenait à lire chacun des caractères deux fois et donc de doubler la complexité temporelle. Il a été alors naturellement convenu de coder notre propre fonction de lecture d'où l'apparition de la fonction statique **`hm_fscanf`** (hm pour home maid). Ainsi nous lisons caractère par caractère dans le fichier donné par la tête de lecture en paramètre grâce à la fonction **`fgetc`**. Elle permet de gérer en une seule fois le cas où nous devons prendre en compte la ponctuation ou non.

6 Performances

Analyse de quelques données : Nous allons analyser quelques données afin d'avoir une idée de la performance de notre exécutable, nous verrons ensuite une étude théorique afin de conformer nos observations, enfin nous donnerons quelques pistes d'améliorations.

Avec le module `holdall` implémenté par LDSC:

Fichiers	Tailles (MB)	Nombre de mots	Temps réel (s)	Allocations (bytes)	Distance de Jaccard
abeeeillmrsss / lesmiserables.txt	3.1 / 3.1	60242 / 60242	2.557	13062070	0
tartuffe.txt / lesmiserables.txt	0.1163 / 3.1	4804 / 60242	1.366	7002984	0.9474
tartuffe.txt / domjuan.txt	0.1163 / 0.1007	4804 / 4255	0.077	743102	0.7852
fr_.txt / domjuan.txt	3.9 / 0.1007	336528 / 4255	1.165	23098730	0.9957

Avec le module holdall implémenté par tableaux dynamiques avec gestion préfixielle:

Fichiers	Tailles (MB)	Nombre de mots	Temps réel (s)	Allocations (bytes)	Distance de Jaccard
abeeeillmrsss / lesmiserables.txt	3.1 / 3.1	60242 / 60242	38.538	12707198	0
tartuffe.txt / lesmiserables.txt	0.1163 / 3.1	4804 / 60242	11.679	6486544	0.9474
tartuffe.txt / domjuan.txt	0.1163 / 0.1007	4804 / 4255	0.287	1122454	0.7852
fr_.txt / domjuan.txt	3.9 / 0.1007	336528 / 4255	96.309	25510530	0.9957

Sans le module holdall en libérant les ressources avec la fonction bst_dft_infix_apply_context du module avl :

Fichiers	Tailles (MB)	Nombre de mots	Temps réel (s)	Allocations (bytes)	Distance de Jaccard
abeeeillmrsss / lesmiserables.txt	3.1 / 3.1	60242 / 60242	0.848	11134310	0
tartuffe.txt / lesmiserables.txt	0.1163 / 3.1	4804 / 60242	0.424	5962232	0.9474
tartuffe.txt / domjuan.txt	0.1163 / 0.1007	4804 / 4255	0.023	598142	0.7852
fr_.txt / domjuan.txt	3.9 / 0.1007	336528 / 4255	0.393	17646186	0.9957

Conclusion : Il est évident que la version sans holdall est bien plus performante.

Précisions sur les résultats obtenus:

Avec LDSC : Comme nous pouvons le remarquer dans le premier tableau, les statistiques temporelles sont relativement satisfaisantes lorsque nous les comparons avec celles relevant d'une implémentation par tableaux dynamiques. Ce qui pose problème ici, c'est le nombre d'allocations. En effet, ce nombre est le plus élevé des trois implémentations ci-dessus. Ce qui est cohérent puisque nous allouons une cellule de liste qui contient la référence et un champ `next` pour chaque mot trouvé dans les fichiers. Nous pouvons donc l'améliorer en essayant de réduire le nombre d'allocations.

Par tableaux dynamiques : Ce qui, à première vue, semblait une amélioration possible de notre projet, c'est avérée la pire implémentation de celles proposées dans notre projet. Nous avons effectivement réduit le nombre d'allocations par rapport au LDSC, mais nous perdions un temps considérable à recopier le tableau dynamique lorsque celui-ci était trop petit. Comme nous pouvons le voir nous avons environ 39 secondes pour traiter les fichiers `abeeeillmrsss.txt` et `lesmiserables.txt`. Nous ne pouvions pas rester sur cette implémentation.

Sans module `Holdall` : En effet supprimer totalement ce module permettrait de supprimer beaucoup d'allocations et donc de gagner du temps. De plus nous stockons déjà toutes les références des mots dans les "AVL fichiers", nous avons donc utilisé la fonction `bst_dft_infix_apply_context` pour nous permettre de parcourir tous les AVL et ainsi libérer les références. Et cette méthode, c'est avérée la meilleure au vu des résultats. Avec les mêmes textes cités précédemment nous avons pu diviser le temps par 45 par rapport à l'implémentation par tableau dynamique, et environ 2M d'octets de gagnés par rapport au LDSC.

7 Améliorations possibles

Une implémentation totalement différente : Nous avons remarqué grâce à l'exécutable fourni sur universitice que nous pouvions faire bien mieux en utilisant les tables de hachages. En effet, en utilisant une table de hachage qui contiendrait tous les mots distincts en tant que clés et l'appartenance dans les fichiers en tant que valeurs, nous aurions eu besoin d'allouer seulement la table de hachage ; là où, pour notre projet, nous avons besoin d'un AVL par fichiers et également un AVL d'union totale pour l'affichage du graphe lorsque celui-ci est demandé. Nous perdons donc beaucoup de temps à allouer les ressources nécessaires.

N'allouer qu'une seule fois les références : Un autre problème de notre implémentation, c'est vu qu'on crée un AVL par fichier ce qui cause plusieurs allocations d'un même mot lorsque celui-ci existe dans plusieurs fichiers. La solution serait de stocker des pointeurs vers les références, qui elles sont uniques, dans les AVL. Nous avons déjà pensé à cette solution avant mais ne pouvions pas le coder vu les restrictions données dans le sujet, c'est-à-dire :

"vous utiliserez nécessairement soit l'en-tête `hashtable.h`, soit l'en-tête `bst.h`, sans les modifier"